

A
Project Report on

Customer Review Sentiment Analytics

Big Data Analytics using Python and Machine Learning

Prepared For
Big Data Analytics (BDA)

Mini Project · SDG 9 and SDG 12

Submitted by
Bharath Rathinasapabathy (123A8012)

SIES GRADUATE SCHOOL OF TECHNOLOGY NERUL,
NAVI MUMBAI 400706
Department of Artificial Intelligence & Data Science
ACADEMIC YEAR 2025-26

A
Report on

Customer Review Sentiment Analytics

Big Data Analytics using Python and Machine Learning

Prepared For
Big Data Analytics (BDA) · Mini Project

Submitted by: Bharath Rathinasapabathy (123A8012)

Department of Artificial Intelligence & Data Science

Under the guidance of

SIES GRADUATE SCHOOL OF TECHNOLOGY NERUL,
NAVI MUMBAI 400706
ACADEMIC YEAR 2025-26

Acknowledgement

I express my sincere gratitude to my Big Data Analytics course instructor for the direction, feedback and encouragement that shaped this project, and for the freedom to turn a classroom topic into a working end-to-end system.

I thank the Department of Artificial Intelligence & Data Science and SIES Graduate School of Technology, Nerul, for the laboratory facilities and the learning environment.

I also acknowledge the open-source communities behind Python, Pandas, NumPy, SciPy, scikit-learn, Jupyter, Plotly and Streamlit, and the authors of the Multilingual Amazon Reviews Corpus and the Hugging Face dataset hub, whose openly available data made a realistic big-data experiment possible.

Bharath Rathinasapabathy

Abstract

Online stores collect thousands of customer reviews every day, and reading them by hand is impossible. This project builds a complete big-data analytics pipeline that reads **199,426 unique Amazon product reviews** and classifies each one as **positive, neutral or negative**, then turns the results into business insight. Star ratings are mapped to sentiment (1-2 stars negative, 3 neutral, 4-5 positive), giving a moderately imbalanced three-class problem (40 % / 20 % / 40 %).

Reviews are cleaned with a custom NLP routine that deliberately **keeps negations and intensifiers** (*not, never, very*), converted to **TF-IDF vectors over unigrams and bigrams** (150,708 features, stored as a sparse matrix that is 99.985% zeros), and used to train and compare six models: a majority-class baseline, Multinomial and Complement Naive Bayes, an SGD classifier, a linear SVM and Logistic Regression. Hyper-parameters are tuned with stratified cross-validation and a validation set, and the selected model is evaluated **once** on a held-out test set of 39,886 reviews.

The final TF-IDF + Logistic Regression model reaches **71.2% accuracy, 0.676 macro-F1 and 0.862 ROC-AUC**, against 40.0% accuracy for the majority-class baseline. Positive and negative reviews are separated well (F1 0.81 and 0.76); the neutral class is the hardest (F1 0.45), and 86% of all errors involve it, because 3-star reviews mix praise and complaints.

The big-data dimension is addressed with sparse-matrix processing, a scalability study (accuracy rises from 65.6% to 71.2% as training data grows from 3,190 to 159,540 reviews, with near-linear training time) and an **out-of-core streaming** variant that learns from 20,000-review chunks using feature hashing and incremental SGD (71.4% accuracy without ever loading the whole dataset, after an out-of-core shuffle fixed a collapse to 40.0% caused by the sorted input file). A keyword-theme analysis shows that customer service, product quality and shipping are the topics most associated with unhappy customers. The system is delivered as a reproducible code base with 30 automated tests, an executed Jupyter notebook and an interactive **Streamlit dashboard** with single-review prediction, word-level explanations, batch CSV analysis and a performance dashboard.

Keywords: big data analytics, sentiment analysis, text classification, TF-IDF, logistic regression, sparse matrices, out-of-core learning, feature hashing, scikit-learn, Streamlit.

Table of Contents

Acknowledgement.....	3
Abstract	4
Chapter 1: Introduction.....	8
1.1 Background.....	8
1.2 Problem Statement	8
1.3 Motivation	8
1.4 Objectives	8
1.5 Alignment with Sustainable Development Goals	9
1.6 Scope and Limitations	9
1.7 Organisation of the Report.....	9
Chapter 2: Literature Review and Technical Background	10
2.1 Sentiment Analysis	10
2.2 Lexicon-based and Machine-learning Approaches	10
2.3 Text Representation: Bag of Words and TF-IDF	10
2.4 Classifiers for Text	10
2.5 Big-data Techniques for Text.....	10
2.6 Imbalanced Classification and Evaluation	11
2.7 Research Gap and Contribution	11
Chapter 3: Requirements and Problem Analysis.....	12
3.1 Stakeholders and Their Needs.....	12
3.2 Functional Requirements	12
3.3 Non-Functional Requirements	13
3.4 Why This Is a Big-data Problem	13
Chapter 4: System Design.....	14
4.1 Architecture Overview.....	14
4.2 Layers and Responsibilities.....	14
4.3 Data and Artefact Files	15
4.4 Evaluation Protocol	15
4.5 Dashboard Design.....	15
4.6 Privacy and Security Considerations	16
Chapter 5: Machine-learning Methodology.....	17
5.1 Dataset	17
5.2 Text Cleaning	17

5.3 Feature Engineering: TF-IDF with n-grams	18
5.4 Classifiers	18
5.5 Handling Class Imbalance	18
5.6 Evaluation Metrics	19
5.7 Hyper-parameter Tuning	19
5.8 Explainability	19
5.9 Worked Example	19
Chapter 6: Big-data Techniques	21
6.1 Sparse Representation	21
6.2 Scalability Study	21
6.3 Out-of-core (Streaming) Training	21
6.4 Path to a Cluster	22
Chapter 7: Implementation	23
7.1 Tools and Technologies	23
7.2 Project Structure	23
7.3 Dashboard Screens	23
Chapter 8: Testing and Results	34
8.1 Test Strategy	34
8.2 Exploratory Data Analysis	34
8.3 Model Comparison and Tuning	35
8.4 Ablation Study: Do the Design Choices Matter?	36
8.5 Final Test-set Results	37
8.6 Error Analysis	38
8.7 What Did the Model Learn?	39
8.8 Business Insight: Complaint Themes	39
8.9 Performance	40
8.10 Threats to Validity and Limitations	40
Chapter 9: Social Impact, Ethics and Sustainability	42
9.1 Expected Impact	42
9.2 Ethical Considerations	42
9.3 Sustainability	42
Chapter 10: Mapping to Programme Outcomes and Skills	42
10.1 Big-data Concepts Demonstrated	42
10.2 Programme Outcome Mapping	43

Chapter 11: Conclusion and Future Scope	44
11.1 Conclusion	44
11.2 Future Scope.....	44
References.....	44
Appendix A: How to Run the Project.....	46
Appendix B: Key Source Code.....	46

Chapter 1: Introduction

1.1 Background

Customer reviews have become one of the most valuable and most under-used data sources of modern e-commerce. A single popular product can collect tens of thousands of reviews, and a marketplace collects hundreds of millions. Buyers use them to decide what to purchase, and sellers use them to learn what to fix. Yet the volume makes manual reading impossible, and star ratings alone hide the reasons behind a score.

Sentiment analysis, the automatic detection of opinion polarity in text [1][2], turns this unstructured text into numbers that can be counted, trended and acted upon. Doing it well at scale is a big-data problem: the data are large, they arrive continuously, they are noisy, and the useful signal is buried in a very high-dimensional, very sparse representation. This project treats customer-review sentiment as a complete analytics workflow, from raw data to a deployed dashboard.

1.2 Problem Statement

Classify product reviews as positive, negative or neutral using Python and machine learning, and extract insight from the results, using techniques that remain practical as the number of reviews grows.

1.3 Motivation

- **Scale.** Hundreds of thousands of reviews cannot be read manually; an automatic classifier gives an instant, consistent first reading.
- **Beyond star ratings.** The words in a review explain *why* a customer is happy or unhappy, which a number cannot.
- **Business value.** Product teams need to know which themes (delivery, quality, support) drive dissatisfaction.
- **A true big-data workload.** Text produces millions of sparse features; handling it exercises sparse linear algebra, scalable learners and out-of-core processing.
- **Explainability.** A black-box label is hard to trust; showing which words drove a decision builds confidence and helps debugging.

1.4 Objectives

1. Collect and profile a large, real review dataset (volume, quality, class balance).
2. Clean the text with an NLP routine that preserves sentiment-bearing words such as negations.
3. Convert text to numerical features with TF-IDF over words and bigrams, stored as sparse matrices.
4. Train and compare several machine-learning classifiers with a rigorous train / validation / test protocol.
5. Handle class imbalance and report appropriate metrics (macro-F1, per-class precision and recall, ROC-AUC).
6. Analyse errors and explain predictions at word level.

7. Demonstrate scalability: learning curve, training-time growth and out-of-core streaming training.
8. Extract business insight (complaint themes) from the review text.
9. Deliver a Jupyter notebook, a reproducible code base with automated tests, and an interactive Streamlit dashboard.

1.5 Alignment with Sustainable Development Goals

The project supports two Sustainable Development Goals [15]. The table lists the targets that the work contributes to.

Table 1: Sustainable Development Goals addressed by the project

Goal	Target (summary)	Contribution of this project
SDG 12	12.6, 12.8: encourage companies to adopt sustainable practices; ensure people have relevant information for responsible choices.	Turns consumer feedback into measurable signals about product quality and durability, helping producers reduce faulty products, returns and waste.
SDG 9	9.5, 9.b: enhance scientific research and upgrade technological capabilities; support domestic technology development.	Demonstrates an open-source, laptop-scale data-science pipeline that small businesses can reuse without expensive infrastructure.
SDG 8	8.2: achieve higher productivity through technological upgrading and innovation.	Automates a manual reading task, freeing analysts to act on insights.

1.6 Scope and Limitations

In scope: English-language product reviews, three-class sentiment classification, classical machine-learning models on TF-IDF features, scalability experiments on a single machine, interactive dashboard, and keyword-based theme analysis.

Out of scope: multilingual text, aspect-based sentiment, deep-learning / transformer models, a real cluster deployment (Spark), and live ingestion from a store's API. These are listed as future work in Chapter 11.

1.7 Organisation of the Report

Chapter 2 reviews related work and the techniques used. Chapter 3 analyses requirements and shows why the problem is a big-data problem. Chapter 4 describes the system design. Chapter 5 explains the machine-learning methodology and Chapter 6 the big-data techniques. Chapter 7 covers the implementation and user interface. Chapter 8 presents testing and results. Chapter 9 discusses social impact and ethics, Chapter 10 maps the work to programme outcomes, and Chapter 11 concludes with future scope.

Chapter 2: Literature Review and Technical Background

2.1 Sentiment Analysis

Sentiment analysis, also called opinion mining, studies the computational treatment of opinions and attitudes in text [1][2]. Early work framed it as text classification: Pang and Lee showed that machine-learning classifiers outperform hand-written rules on movie reviews [1]. Tasks differ in granularity (document, sentence or aspect level) and in the label scheme (binary, three-class, or a five-star scale). This project works at document level with three classes.

2.2 Lexicon-based and Machine-learning Approaches

Lexicon-based methods score a text by looking up each word in a dictionary of sentiment values. VADER is a well-known rule-based example tuned for social-media text [11]. They need no training data but cannot adapt to a domain: the word *cheap* is praise in a price review and criticism in a quality review. Supervised machine-learning methods learn these domain associations from labelled examples, which is why this project trains classifiers on star-labelled reviews.

2.3 Text Representation: Bag of Words and TF-IDF

A classifier needs numerical input. The bag-of-words model represents a document by the counts of the words it contains. TF-IDF [3] re-weights the counts so that a term scores highly when it is frequent in a document but rare across the collection, which suppresses uninformative words and emphasises distinctive ones. Adding **n-grams** (sequences of 2 or more words) lets a linear model capture short phrases such as *not good* or *stopped working* that single words cannot express.

2.4 Classifiers for Text

- **Naive Bayes** [5] applies Bayes' rule with a conditional-independence assumption. It is extremely fast and a strong baseline for text; the Complement variant corrects for class imbalance.
- **Logistic Regression** models class probabilities with a softmax over a linear score. It is well calibrated, interpretable (each word has a weight per class) and scales to millions of sparse features [20].
- **Support Vector Machines** [6] find the maximum-margin hyper-plane. Joachims showed that linear SVMs are particularly effective on high-dimensional sparse text.
- **Stochastic Gradient Descent (SGD) classifiers** [8] optimise a linear model one example (or mini-batch) at a time, which makes them suitable for very large data and for streaming.
- **Transformer models** such as BERT [9] and DistilBERT [10] reach higher accuracy but need far more compute and, ideally, GPUs; they are listed as future work.

2.5 Big-data Techniques for Text

Text data stress memory more than they stress the processor. A matrix of 40,000 reviews by 150,000 features would need about 24 GB if stored densely, yet each review uses only a few dozen words. **Sparse matrix** formats store only the non-zero entries. When the vocabulary itself is too large to hold in memory, the **hashing trick** [7] maps each word to one of a fixed number of buckets, removing the need

for a vocabulary. Combined with incremental learners, this enables **out-of-core learning**, where data are read in chunks and the model is updated after each chunk [8]. For truly distributed workloads, engines such as Apache Spark provide the same operations across a cluster [13].

2.6 Imbalanced Classification and Evaluation

When one class is much rarer, plain accuracy rewards ignoring it [12]. Macro-averaged F1 gives each class equal weight, and per-class precision and recall show where errors occur. For ranking quality, the area under the ROC curve [14] is computed one-vs-rest for multi-class problems. A strict train / validation / test protocol prevents over-optimistic estimates caused by tuning on the test data.

2.7 Research Gap and Contribution

Table 2: Gaps addressed by this work

Gap in typical coursework projects	How this project addresses it
Small toy datasets (a few hundred reviews)	199,426 real reviews, a genuine big-data workload.
Binary labels only; neutral ignored	Three classes with explicit analysis of why neutral is hard.
Standard stop-word removal destroys negation	Custom stop-word list keeps negations; an ablation study proves the benefit.
Only accuracy reported	Macro-F1, per-class metrics, ROC-AUC, confusion matrix and error analysis.
Model chosen on the test set	Separate validation set for selection; test set used once.
No evidence the method scales	Learning-curve, timing study and out-of-core streaming training.
Notebook only	Reusable modules, automated tests and an interactive dashboard with explanations.

Chapter 3: Requirements and Problem Analysis

3.1 Stakeholders and Their Needs

Table 3: Stakeholders and their needs

Role	Needs	What the system gives
Product / business manager	Know what customers like and dislike, and whether sentiment is changing.	Sentiment mix, complaint-theme ranking and trends in the Insights page.
Customer-support lead	Spot unhappy customers quickly.	Batch analysis flags negative reviews with confidence scores; CSV export.
Data analyst	Trust and audit the model.	Notebook with full experiments, metrics, error analysis and word-level explanations.
Student / evaluator	Reproduce results.	One-command training, tests, fixed random seeds and documented design decisions.

3.2 Functional Requirements

Table 4: Functional requirements

ID	Requirement	Module
FR1	Load the review dataset, remove empty and duplicate reviews, derive sentiment from star ratings.	src/preprocess.py
FR2	Clean text (HTML, URLs, case, contractions) while preserving negations and intensifiers.	src/preprocess.py
FR3	Convert reviews to TF-IDF unigram + bigram features as sparse matrices.	src/modeling.py
FR4	Train and compare at least five classifiers plus a baseline.	src/train.py, notebook
FR5	Tune hyper-parameters using cross-validation and a validation set.	src/train.py, notebook
FR6	Evaluate once on a held-out test set: accuracy, precision, recall, F1, ROC-AUC, confusion matrix.	src/train.py
FR7	Persist the trained pipeline and all metrics for reuse.	models/ outputs/metrics.json
FR8	Predict the sentiment of one review with class probabilities.	app.py (Analyze)
FR9	Explain a prediction by highlighting the words that drove it.	app.py (Analyze)
FR10	Classify an uploaded CSV of reviews and let the user download the results.	app.py (Batch)
FR11	Display model performance, model comparison, scalability and error analysis.	app.py (Performance)
FR12	Provide exploratory insight: class balance, review length, frequent phrases, complaint themes.	notebook, app.py (Insights)
FR13	Demonstrate out-of-core training on streamed chunks.	notebook
FR14	Provide automated tests and a single command to retrain.	tests/, python -m src.train

3.3 Non-Functional Requirements

- **Performance:** single-review prediction in well under 100 ms (measured median 4.4 ms); batch throughput of thousands of reviews per second.
- **Reproducibility:** fixed random seeds, stratified splits, pinned dependency list and a single training command.
- **Usability:** a clean dashboard that needs no code; sample files and one-click examples.
- **Explainability:** every prediction can be justified with the words that caused it.
- **Portability:** runs on an ordinary laptop with open-source libraries only; no GPU.
- **Maintainability:** modular code (preprocess, modeling, train, facts, app) with automated tests.
- **Privacy:** reviews are public product reviews; uploaded CSVs are processed in memory and not stored.

3.4 Why This Is a Big-data Problem

Big data is usually described by the five Vs. The table shows how each applies to review analytics.

Table 5: The five Vs applied to the project

V	Meaning	In this project
Volume	Large amount of data	199,426 reviews, about 47 MB of raw text; the TF-IDF matrix has 150,708 columns, which would need 24 GB if dense for the test set alone.
Velocity	Data arrive continuously	Stores receive new reviews every minute; the streaming trainer processes data chunk by chunk as they arrive.
Variety	Many forms and formats	Free text with HTML, URLs, emoji, typos, mixed length (1 to 769 words) and informal grammar.
Veracity	Quality and trustworthiness	Star ratings are noisy proxies for sentiment; duplicates, mis-clicked ratings and sarcasm exist (Section 8.7).
Value	Business insight extracted	Sentiment mix and complaint themes tell product teams where to improve.

Chapter 4: System Design

4.1 Architecture Overview

The system has three layers. The **data layer** loads, cleans and splits the reviews, and can also stream them in chunks. The **machine-learning layer** cleans text, builds TF-IDF features, compares and tunes models, evaluates the winner and persists it together with its metrics. The **presentation layer** consists of the Jupyter notebook (full analysis) and the Streamlit dashboard (interactive use). The layers communicate only through files, which keeps them independent and reproducible.

System architecture

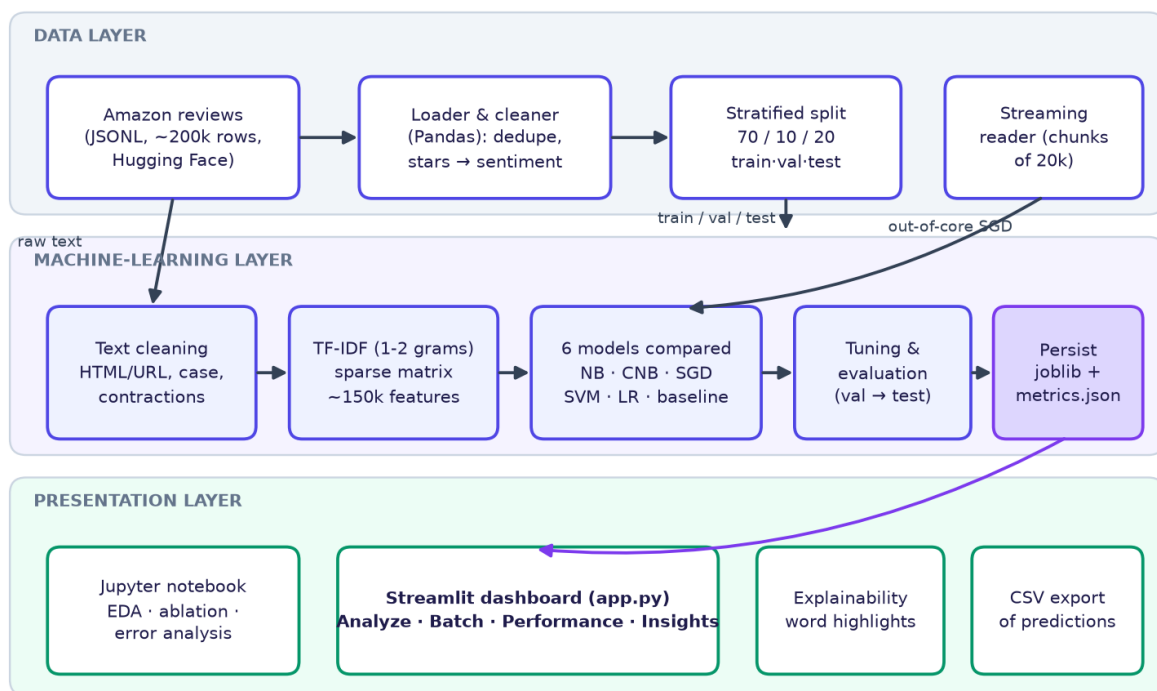


Figure 1: System architecture of the sentiment analytics project

4.2 Layers and Responsibilities

Table 6: Layers and responsibilities

Layer	Component	Responsibility
Data	src/preprocess.py	Load JSONL, drop empty/duplicate reviews, map stars to sentiment, clean text.
Data	src/download_data.py	Fetch the 47 MB dataset so the repository stays small.
ML	src/modeling.py	TF-IDF vectoriser factory and scoring helpers shared by script, notebook and app.
ML	src/train.py	Compare six models, tune, refit, evaluate on test, save model, figures and metrics.
ML	src/facts.py	Measure latency, throughput, sparse-matrix memory and the error breakdown.
Presentation	notebooks/*.ipynb	EDA, ablation, grid search, error analysis, scalability and streaming experiments.

Layer	Component	Responsibility
Presentation	app.py + assets/style.css	Streamlit dashboard with six pages and custom styling.
Quality	tests/	Unit tests, model sanity tests and dashboard smoke tests.

4.3 Data and Artefact Files

Table 7: Data and artefact files

File	Content	Produced by
data/amazon_reviews_raw.jsonl	Raw reviews (id, text, label 0-4); 200,000 rows	src/download_data.py
data/sample_reviews.csv	200 held-out reviews with true labels (demo)	src/train.py
data/demo_reviews.csv	42 hand-written reviews across 14 products (demo)	authored
models/sentiment_model.joblib	Fitted TF-IDF + Logistic Regression pipeline (5.3 MB)	src/train.py
outputs/metrics.json	All metrics, confusion matrix, ROC points, top words, EDA statistics	src/train.py
outputs/*.csv, streaming.json	Ablation, scalability, themes, error examples, streaming result	notebook
outputs/facts.json	Latency, throughput, memory, error breakdown	src/facts.py
outputs/figures/*.png	All charts used in the notebook, app and this report	src/train.py, notebook

4.4 Evaluation Protocol

A stratified split keeps the 40 / 20 / 40 class ratio in every part. Model comparison and tuning use only the validation set; the final model is refit on train plus validation and scored exactly once on the test set.

Evaluation protocol (stratified split of 199,426 reviews)

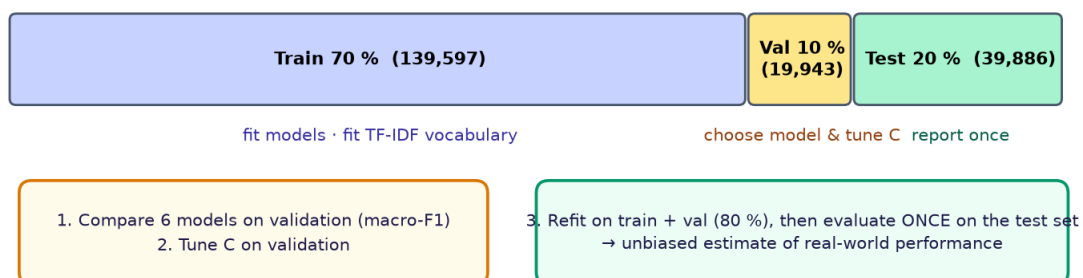


Figure 2: Train / validation / test protocol

4.5 Dashboard Design

The Streamlit dashboard has six pages reachable from a sidebar: Overview, Analyze Review, Batch Analysis, Model Performance, Data Insights and About. Charts are interactive Plotly figures; colours are consistent (red = negative, amber = neutral, green = positive) so that a viewer can read any chart at a glance. The model and metrics are cached, so every interaction after the first is instantaneous. Pages

can be opened directly with URL parameters (for example `?page=analyze&example=negative`), which was also used to capture the screenshots in this report.

4.6 Privacy and Security Considerations

- The dataset contains public product reviews without account identifiers.
- Uploaded CSV files are read into memory for the session only; nothing is written to disk or sent to external services.
- The model file is loaded from the local repository; no network call is made at prediction time.
- Large uploads are limited by Streamlit's default 200 MB upload cap, and only the first 500 rows are rendered in the table (the full results remain downloadable).

Chapter 5: Machine-learning Methodology

This chapter describes each step from raw review to prediction, the algorithm used and the reason for the design decision.

5.1 Dataset

The data are the English subset of the Multilingual Amazon Reviews Corpus [16], obtained from the Hugging Face hub as `SetFit/amazon_reviews_multi_en`. The training split contains 200,000 reviews, each with a text and a star rating from 1 to 5. The five star levels are almost perfectly balanced at the source (about 40,000 reviews each). After removing 574 duplicate or empty reviews, 199,426 remain. The average review has 34 words; the longest has 769.

Table 8: Sentiment classes derived from star ratings

Class	Stars	Reviews	Share
Negative	1-2	79,819	40.0%
Neutral	3	39,907	20.0%
Positive	4-5	79,700	40.0%

5.2 Text Cleaning

Every review passes through the same deterministic cleaning function, which is also stored inside the saved model so that training and prediction always match.

Table 9: Text-cleaning steps

Step	Operation	Example
1	Decode HTML entities, lower-case	Tom & Jerry → tom & jerry
2	Remove URLs and HTML tags	see http://x.com → see
3	Expand negative contractions	don't → do not; can't → can not; won't → will not
4	Remove other possessive/auxiliary suffixes	it's → it
5	Remove digits and punctuation	10/10 !!! → (empty)
6	Collapse whitespace	a b → a b
7	Remove stop-words except negations and intensifiers	the, and, is removed; not, never, very, too kept

Design decision. Standard stop-word lists (such as scikit-learn's English list) contain *not*, *no*, *never*, *very* and *too*. Removing them turns "*not good*" into "*good*" and flips the meaning. The project therefore removes only genuinely uninformative words and keeps {N_KEPT} negation and intensifier words. Section 8.4 quantifies the effect.

5.3 Feature Engineering: TF-IDF with n-grams

Each cleaned review is converted to a vector of TF-IDF weights over unigrams and bigrams. With sub-linear term frequency, the weight of term t in document d is

$$tfidf(t, d) = (1 + \ln tf(t, d)) \times [\ln((1 + N) / (1 + df(t))) + 1]$$

where tf is the count of t in d , df the number of documents containing t , and N the number of documents. Each vector is then length-normalised (L2). Terms appearing in fewer than 3 reviews or more than 90 % of reviews are discarded, and at most 300,000 features are kept. The final vocabulary has **150,708** terms. The sub-linear scaling stops a word repeated ten times from counting ten times as much, and bigrams capture phrases such as *not good*.

5.4 Classifiers

Logistic Regression (selected model)

A multinomial logistic regression maps the feature vector x to class probabilities with a softmax:

$$P(y = k | x) = \exp(w_k \cdot x + b_k) / \sum_j \exp(w_j \cdot x + b_j)$$

Training minimises the class-weighted cross-entropy plus an L2 penalty controlled by C (a larger C means weaker regularisation). Because each word has one coefficient per class, the model is directly interpretable.

Other models compared

Table 10: Models compared

Model	Idea	Key settings
Majority baseline	Always predict the most frequent class; a floor that any real model must beat.	strategy = most_frequent
Multinomial Naive Bayes	$P(c d) \propto P(c) \prod P(t c)^{tf}$ with Laplace smoothing.	alpha = 0.3
Complement Naive Bayes	Uses statistics of the complement of each class; better for imbalance.	alpha = 0.3
SGD (modified Huber)	Linear model trained by stochastic gradient descent with a smooth margin loss.	alpha = 2e-6, balanced weights
Linear SVM	Maximum-margin separator, loss $\max(0, 1 - y \cdot w \cdot x)$.	$C = 0.1$, balanced weights
Logistic Regression	Softmax over a linear score; gives calibrated probabilities.	C tuned, balanced weights

5.5 Handling Class Imbalance

Neutral reviews make up only 20 % of the data. Balanced class weights multiply the loss of class c by $N / (K \cdot n_c)$, which is about 0.83 for negative and positive and 1.67 for neutral, so that a mistake on a neutral review costs twice as much. Performance is summarised with macro-F1, which averages the F1 of the three classes and cannot be inflated by ignoring the smallest one.

5.6 Evaluation Metrics

Table 11: Evaluation metrics

Metric	Definition	Why it is used
Accuracy	correct / total	Intuitive overall rate; compared against the 40 % baseline.
Precision (class c)	$TP / (TP + FP)$	Of the reviews labelled c, how many truly are c.
Recall (class c)	$TP / (TP + FN)$	Of the true c reviews, how many were found.
F1 (class c)	$2PR / (P + R)$	Balances precision and recall.
Macro-F1	mean of the three class F1 scores	Headline metric; treats every class equally.
ROC-AUC (one-vs-rest)	area under the TPR-FPR curve per class, averaged	Threshold-independent ranking quality [14].
Confusion matrix	counts of actual vs. predicted class	Shows which classes are confused.

5.7 Hyper-parameter Tuning

Tuning happens in two stages, both without touching the test set. First, a 3-fold stratified grid search runs on a 40,000-review subsample with a complete pipeline (the vectoriser is refit inside each fold, so no vocabulary or IDF information leaks from the validation fold). The grid covers n-gram range $\{(1,1), (1,2)\}$, minimum document frequency $\{2, 5\}$ and C $\{0.5, 1, 2, 5\}$. Second, C is fine-tuned on the full training set and scored on the validation set for Logistic Regression and the SVM.

5.8 Explainability

For a review with TF-IDF vector x and predicted class k , the contribution of term j is $x_j \times w_{kj}$, the product of the term's weight in the review and its coefficient for the predicted class. The dashboard colours each word by the class it pushes towards (opacity = strength) and plots the largest contributions, including bigrams such as *stopped working*.

5.9 Worked Example

Table 12: A review passing through the pipeline

Stage	Output
Raw review	I DON'T like it!! Can't recommend, the seller never replied.
After cleaning	i do not like it can not recommend the seller never replied
Predicted class	NEGATIVE (negative 94.8%, neutral 3.9%, positive 1.3%)
Strongest evidence	not recommend (+0.99), not (+0.78), never (+0.61), seller (+0.25), not like (+0.20), seller never (+0.14)

A review flowing through the pipeline (illustrative values)

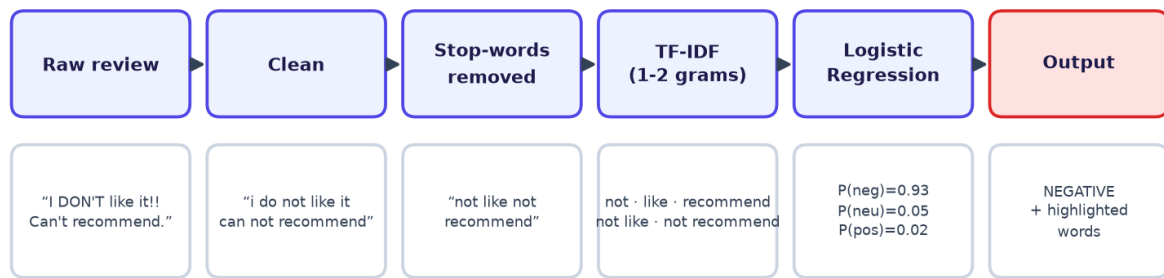


Figure 3: A review flowing through the pipeline

Chapter 6: Big-data Techniques

6.1 Sparse Representation

Text is naturally sparse: a review uses a few dozen of the 150,708 possible terms. For the 39,886-review test set, the TF-IDF matrix has 876,480 non-zero entries (about 22 per review), a sparsity of 99.985%. Stored in compressed sparse row (CSR) format it needs **7 MB**; stored densely it would need **24 GB**, a 3,353-fold difference. Every operation in training (dot products, gradients) runs in time proportional to the number of non-zeros, not to the matrix size, which is why linear models scale well on text.

6.2 Scalability Study

To check how cost and quality grow with data, the final configuration was trained on increasing fractions of the training data and evaluated on the same test set.

Table 13: Scalability study

Training reviews	Accuracy	Macro-F1	Vectorise (s)	Fit (s)
3,190	65.6%	0.6151	0.7	0.3
7,977	67.1%	0.6313	2.6	0.9
15,954	68.3%	0.6434	4.2	1.6
39,885	69.6%	0.6591	8.9	5.1
79,770	70.2%	0.6659	17.4	9.3
159,540	71.2%	0.6757	33.7	37.8

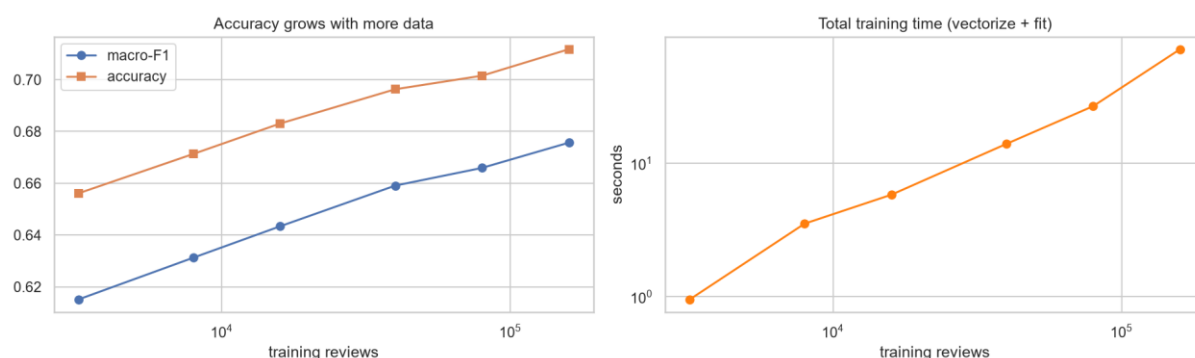


Figure 4: Accuracy rises with more data (left); training time grows close to linearly (right)

Going from 3,190 to 159,540 reviews (a 50-fold increase) lifts accuracy by 5.6 points and macro-F1 by 6.1 points, while vectorising and fitting the whole 160,000-review training set takes about 71 seconds on a laptop. The curve has not flattened, which is the classic argument for analysing reviews at big-data scale: more data still helps.

6.3 Out-of-core (Streaming) Training

When data do not fit in memory, the dataset is processed in chunks. The implementation reads the JSONL file in chunks of 20,000 reviews, converts each chunk with a **HashingVectorizer** (2²⁰ buckets, unigrams and bigrams, no stored vocabulary) and updates an **SGDClassifier** with `partial_fit`.

Reviews from the test set are filtered out of the stream. Memory use depends on the chunk size, not on the file size.

A pitfall discovered during the experiment. Online learners assume that the stream is randomly ordered. The raw dataset file is *sorted by star rating* (every block of 20,000 rows has a single rating, from 1 star first to 5 stars last). A naive single pass therefore ends having seen only positive reviews recently and the model collapses to predicting one class. The fix is an **out-of-core shuffle-shard**: each chunk's rows are scattered at random across 8 shard files (one pass, one chunk in memory at a time), and the shards are then streamed.

Table 14: Out-of-core streaming results

Setting	Value
Reviews streamed	159,885
Chunk size / shards	20,000 / 8
Passes over the data	1
Wall-clock time (shuffle-shard + training)	47 s
Naive stream of the sorted file: accuracy / macro-F1	40.0% / 0.190 (collapsed)
Shuffle-shard stream: accuracy / macro-F1	71.4% / 0.654
In-memory Logistic Regression: accuracy / macro-F1	71.2% / 0.676

After shuffling, a single pass with constant memory reaches 71.4% accuracy, on par with the in-memory model (71.2%), although its macro-F1 is lower (0.654 against 0.676) because the incremental learner is weaker on the neutral class. This is the pattern that scales to millions of reviews. The practical lesson is to always check the order of a stream before training an online learner.

6.4 Path to a Cluster

The same pipeline maps directly onto distributed tools, so the single-machine prototype is a faithful blueprint for a production deployment.

Table 15: Mapping to Apache Spark

This project (scikit-learn)	Distributed equivalent (Apache Spark MLlib) [13]
pandas DataFrame, read_json(chunksize)	Spark DataFrame reading partitioned JSON/Parquet
clean_text() function	UDF or Spark NLP / RegexTokenizer + StopWordsRemover
TfidfVectorizer / HashingVectorizer	Tokenizer + HashingTF + IDF (+ NGram)
LogisticRegression / SGDClassifier	pyspark.ml.classification.LogisticRegression
train_test_split, GridSearchCV	randomSplit, CrossValidator + ParamGridBuilder
joblib model file	Saved Pipeline model, served via MLflow or batch scoring jobs

Chapter 7: Implementation

7.1 Tools and Technologies

Table 16: Tools used

Tool	Version used	Use in the project
Python	3.13	Language for all code.
Pandas	3.0	Loading, cleaning, grouping and chunked reading of reviews.
NumPy / SciPy	2.4 / sparse	Numerical arrays and sparse matrices.
scikit-learn	1.5	TF-IDF, hashing, classifiers, metrics, model selection, pipelines.
Jupyter Notebook	JupyterLab 4.5	Interactive analysis, executed and saved with outputs.
Matplotlib / Seaborn / WordCloud	-	Static figures for the notebook and report.
Plotly	7.x	Interactive charts in the dashboard.
Streamlit	1.60	Dashboard application.
Joblib	1.5	Model persistence.
pytest	-	Automated testing.
Git / GitHub	-	Version control and publication.

7.2 Project Structure

Table 17: Project structure

Path	Purpose
app.py, assets/style.css, .streamlit/config.toml	Dashboard code, stylesheet and theme.
src/preprocess.py, modeling.py, train.py, facts.py, download_data.py	Reusable pipeline modules.
notebooks/Customer_Review_Sentiment_Analysis.ipynb	Full executed analysis (58 cells, 9 embedded figures).
tests/	Unit, model and dashboard tests.
models/, outputs/, data/	Trained model, metrics and figures, datasets.
docs/	This report, the presentation script, diagrams and screenshots.
README.md, requirements.txt, LICENSE, .gitignore	Documentation and repository hygiene.

7.3 Dashboard Screens

The dashboard was designed for clarity and for presentation. The sidebar provides navigation and always shows the headline metrics. All screenshots below were captured from the running application.

Overview

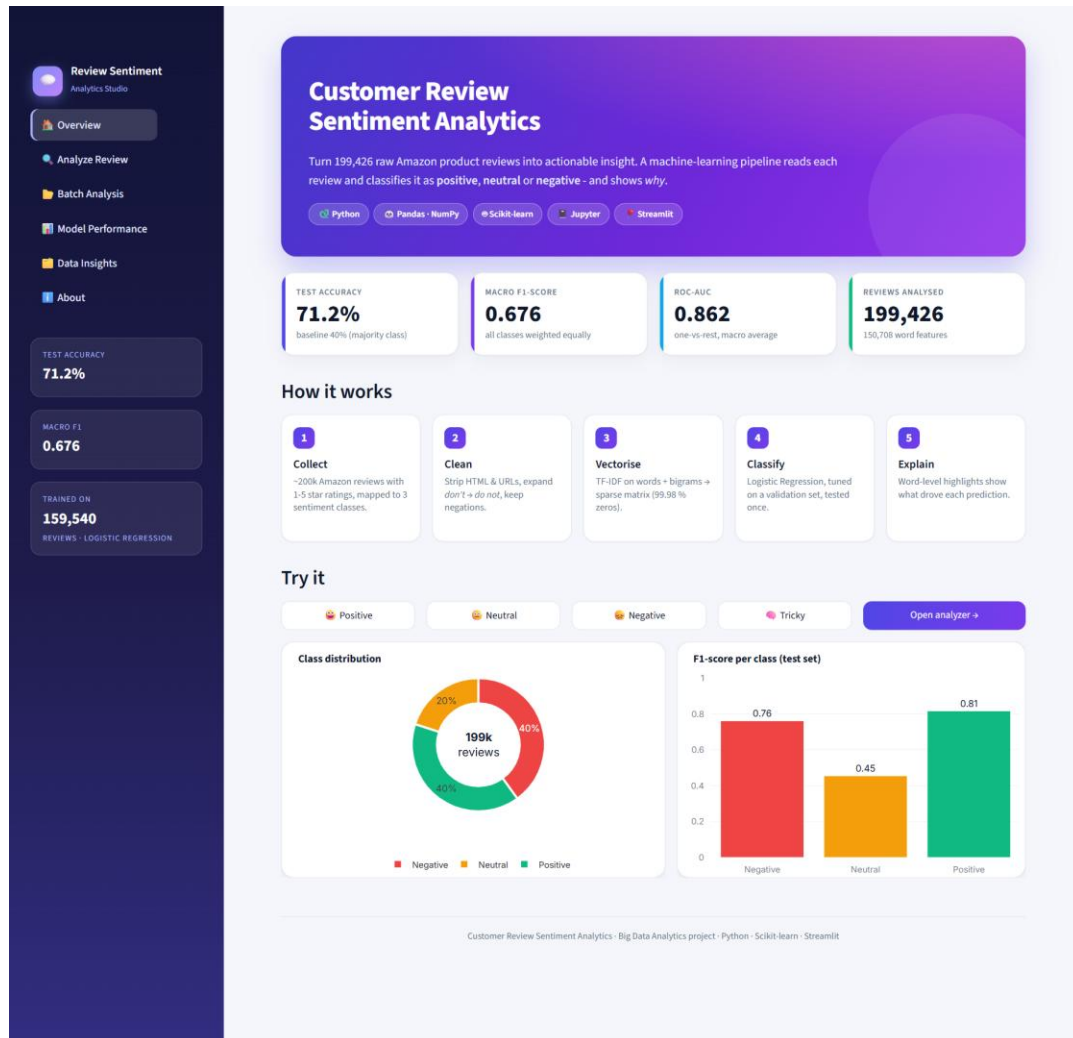


Figure 5: Overview page: headline metrics, the five-step pipeline and quick-start examples

Analyze a review

A user types or pastes a review (or clicks one of four examples, including a tricky negation case). The page shows the predicted class with confidence, probabilities for all three classes, a sentiment meter from -100 to +100, the review with words highlighted by the class they support, and a bar chart of the strongest contributing terms.

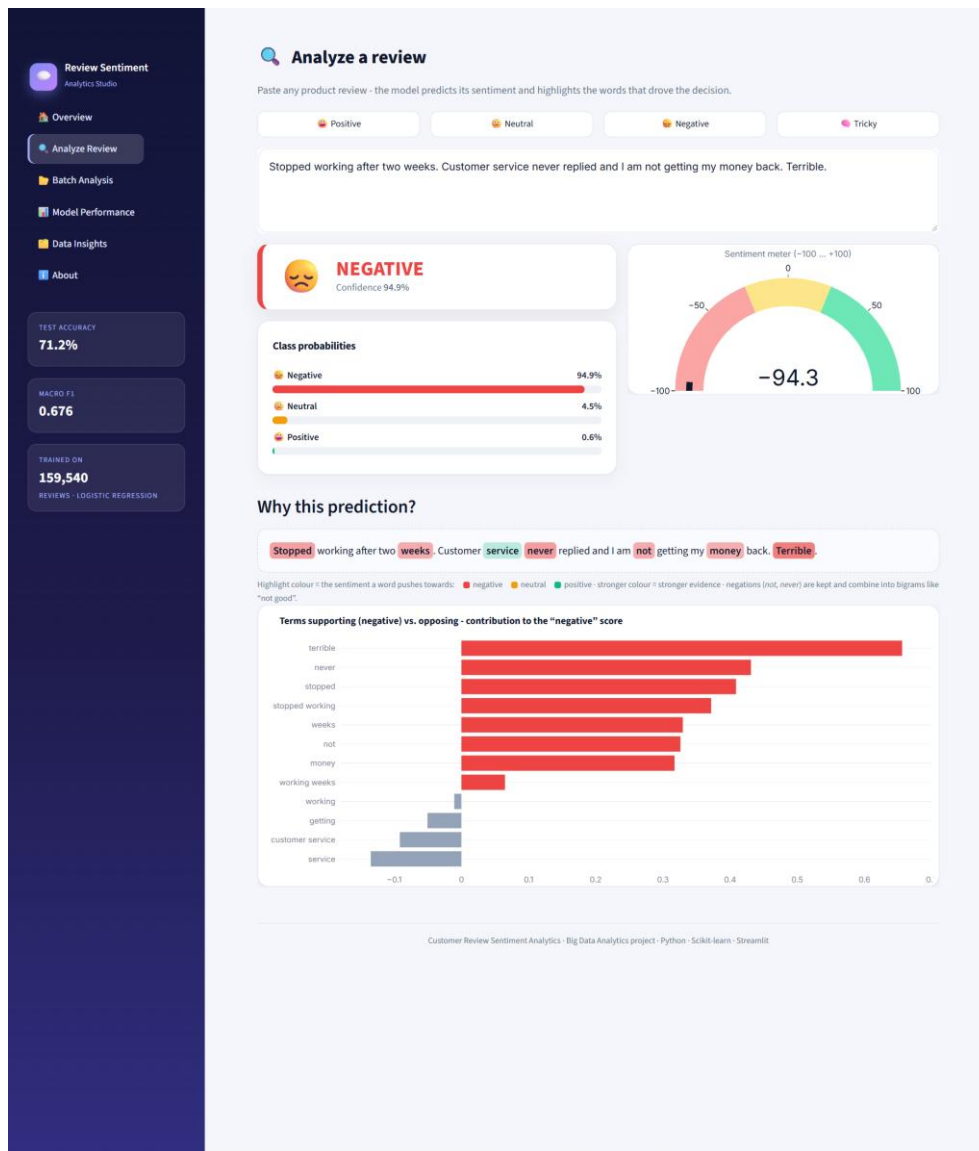


Figure 6: Analyze page: a negative review with highlighted evidence words

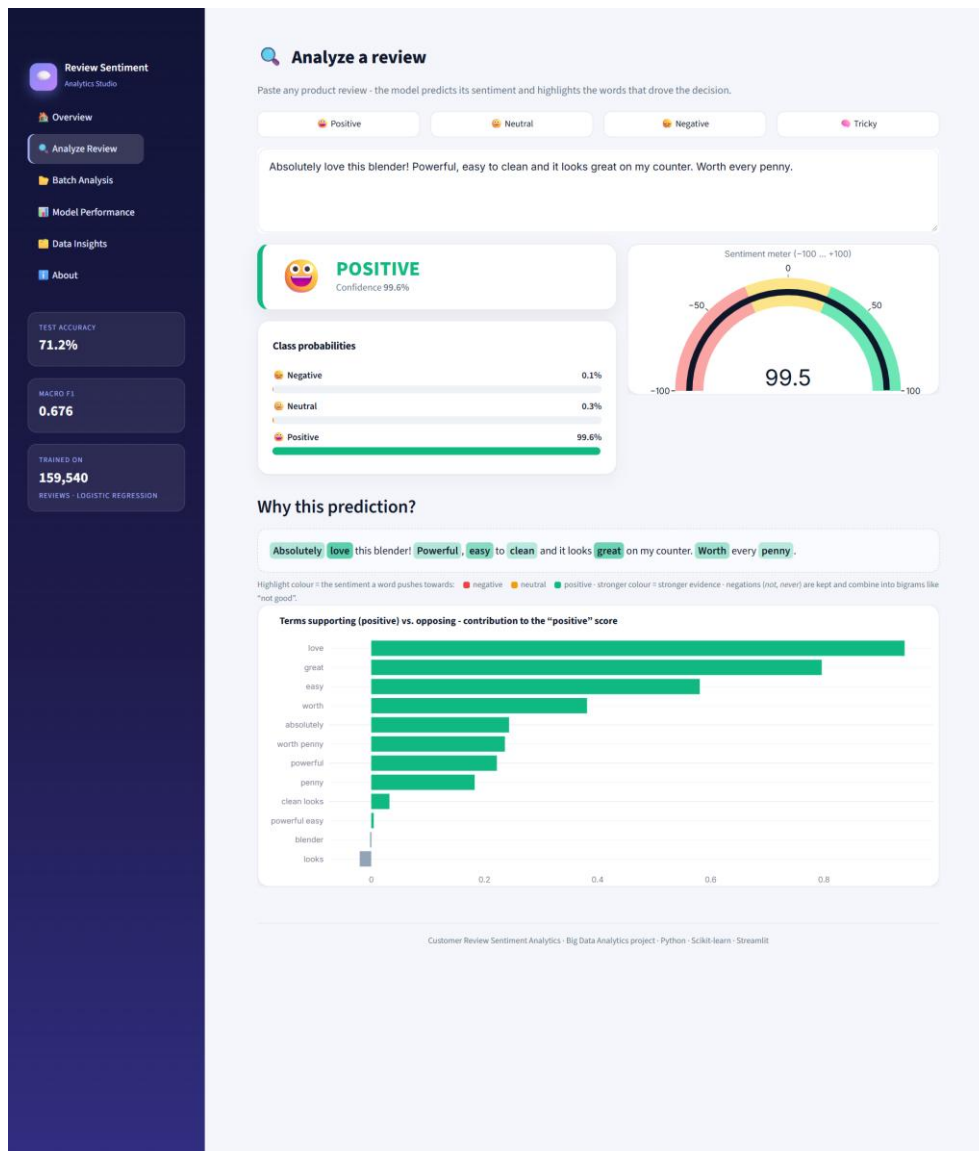


Figure 7: Analyze page: a positive review

Batch analysis

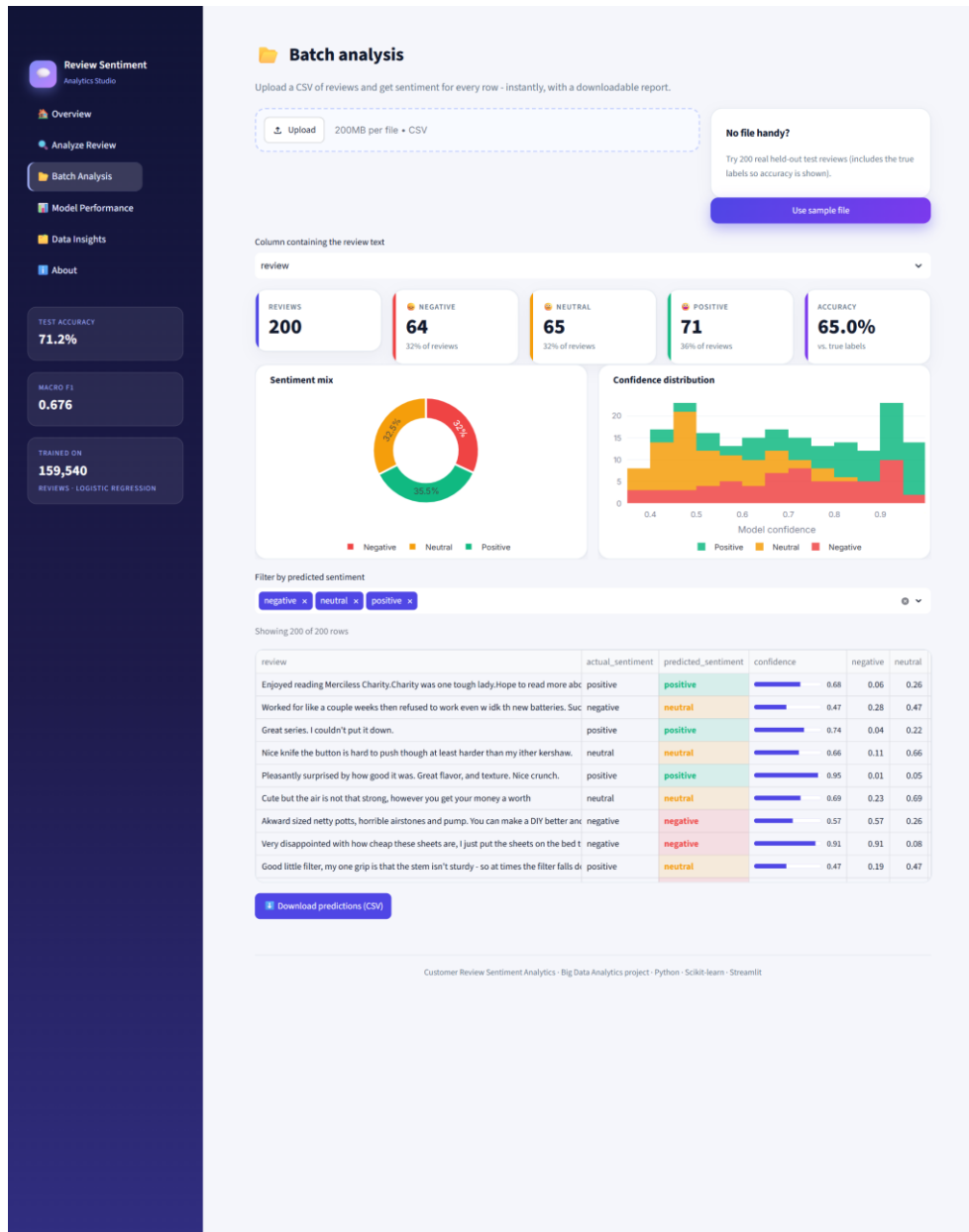


Figure 8: Batch page: 200 sample reviews classified, with sentiment mix, confidence distribution, filterable table and CSV download

Model performance

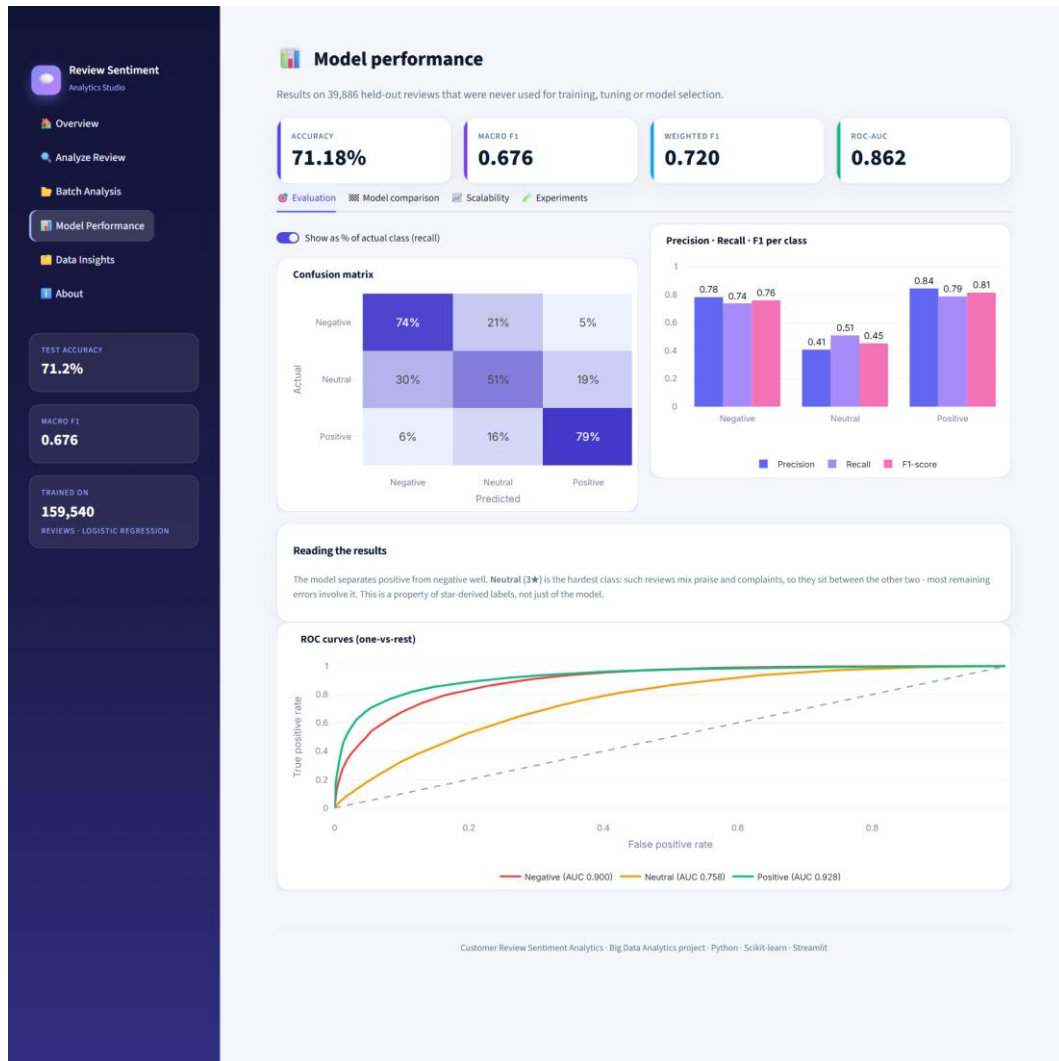


Figure 9: Performance page, Evaluation tab: confusion matrix, per-class metrics and ROC curves

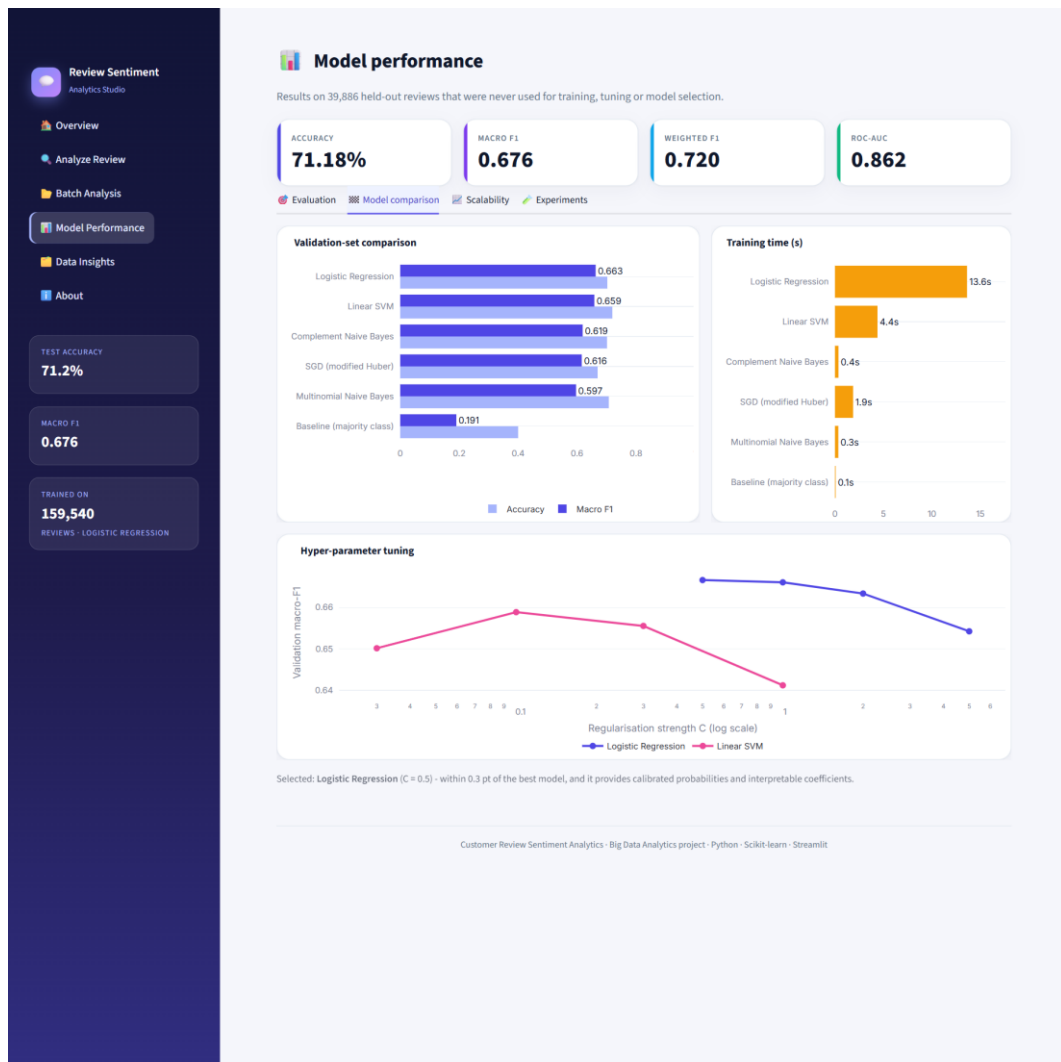


Figure 10: Performance page, Model comparison tab

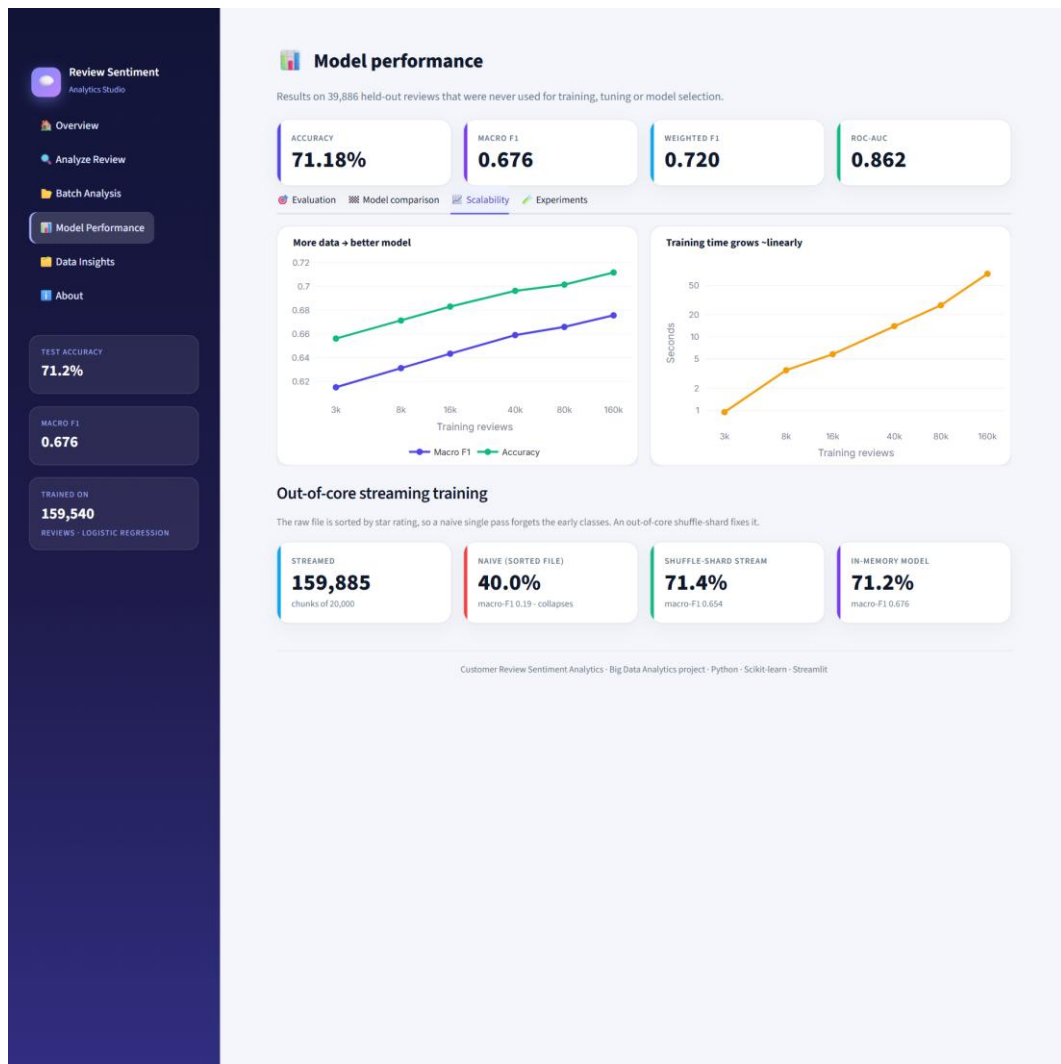


Figure 11: Performance page, Scalability tab: learning curve, timing and streaming result

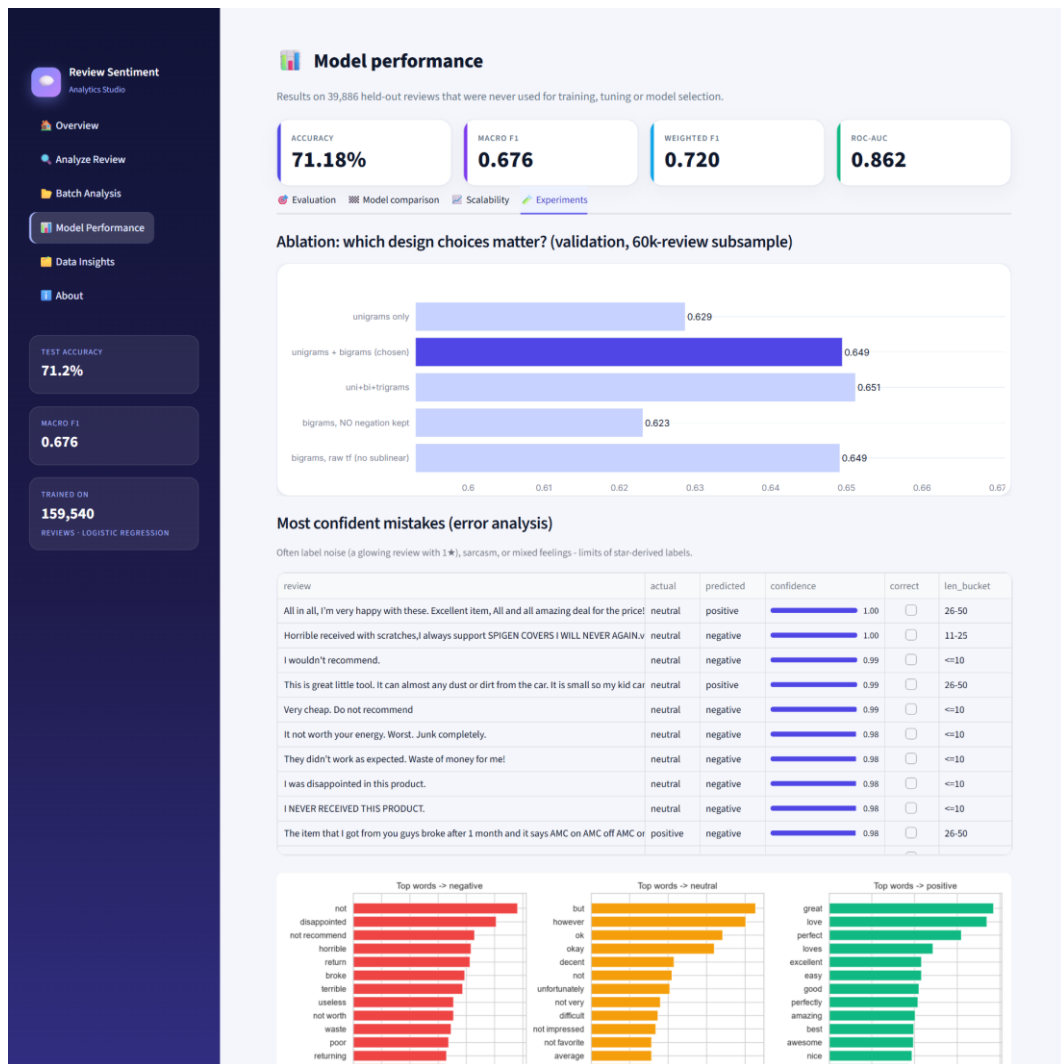


Figure 12: Performance page, Experiments tab: ablation study, confident mistakes and top words

Data insights and About



Figure 13: Insights page: class balance, review length, frequent phrases, complaint themes and word clouds

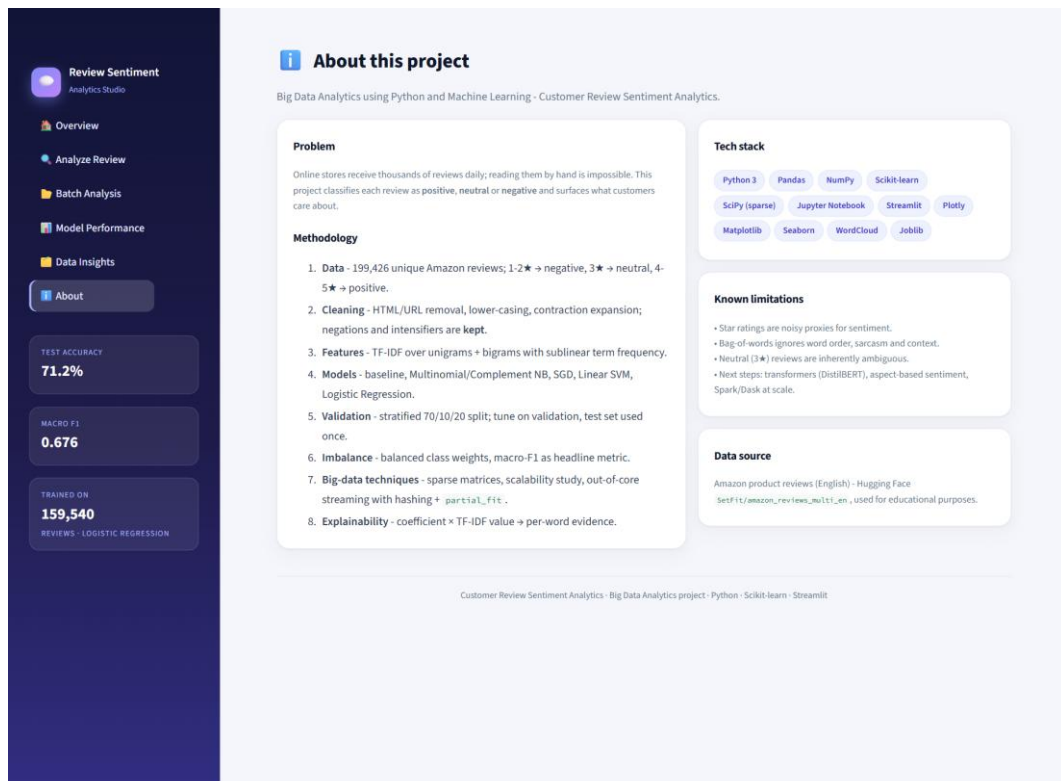


Figure 14: About page: methodology, tech stack and limitations

Chapter 8: Testing and Results

8.1 Test Strategy

Table 18: Automated tests

Test group	What it checks	Tests
tests/test_preprocess.py	Star mapping, cleaning rules, contraction expansion, negation preserved in stop-word list, loader de-duplication.	12
tests/test_model.py	Obvious positive/negative reviews, negation sensitivity, valid probabilities, unseen / non-English / empty text, consistency of metrics file, no ignored class.	10
tests/test_app.py	Every dashboard page renders without exceptions; analyzer predicts negative for a negative review; empty input shows a hint.	8

All **30 tests pass** on the final code (0 failures).

8.2 Exploratory Data Analysis

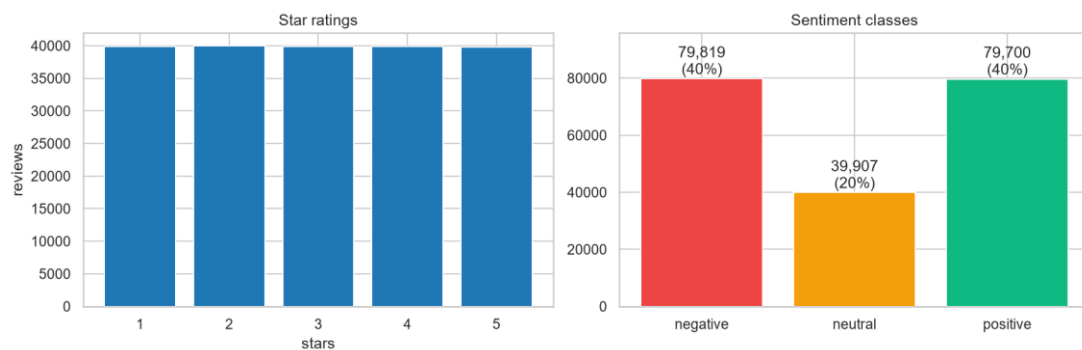


Figure 15: Star ratings (left) and derived sentiment classes (right)

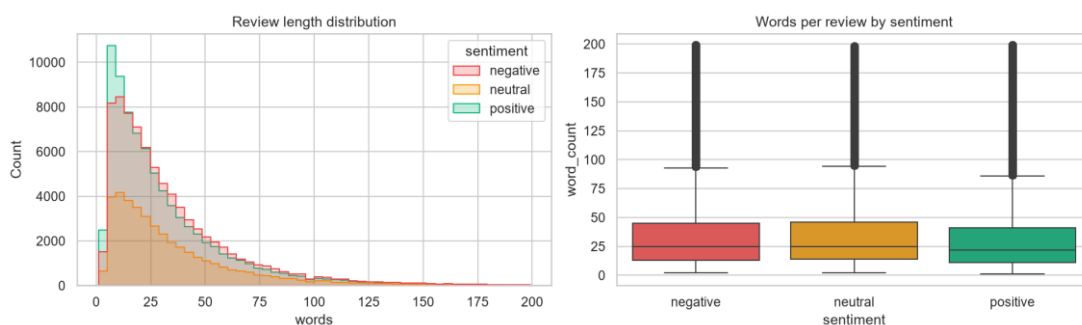


Figure 16: Review length by sentiment

Review length is very similar across classes (mean 35.0 words for negative, 36.4 for neutral and 32.3 for positive), so length alone is a weak signal; the words themselves carry the sentiment. Distributions are strongly right-skewed, with a median of about 25 words.

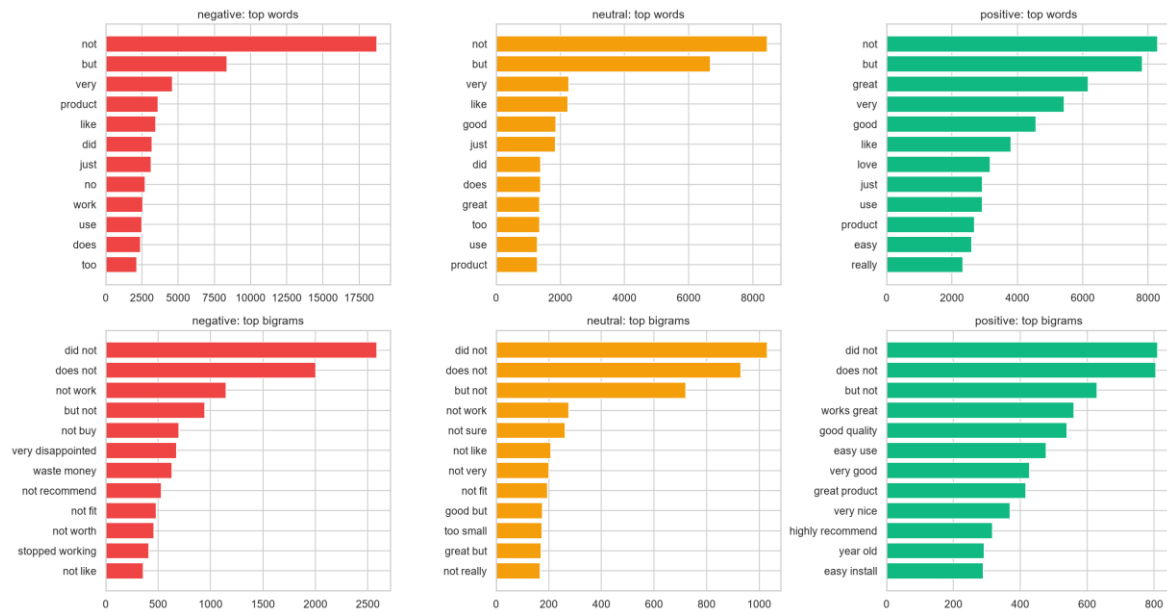


Figure 17: Most frequent unigrams (top row) and bigrams (bottom row) per class



Figure 18: Word clouds per sentiment class

8.3 Model Comparison and Tuning

Table 19: Model comparison on the validation set (19,943 reviews)

Model	Accuracy	Macro-F1	Weighted-F1	Train time (s)
Logistic Regression	70.3%	0.6633	0.7097	13.6
Linear SVM	72.0%	0.6588	0.7121	4.4
Complement Naive Bayes	70.2%	0.6192	0.6815	0.4
SGD (modified Huber)	67.0%	0.6161	0.6692	1.9
Multinomial Naive Bayes	70.8%	0.5968	0.6704	0.3
Baseline (majority class)	40.0%	0.1906	0.2288	0.1

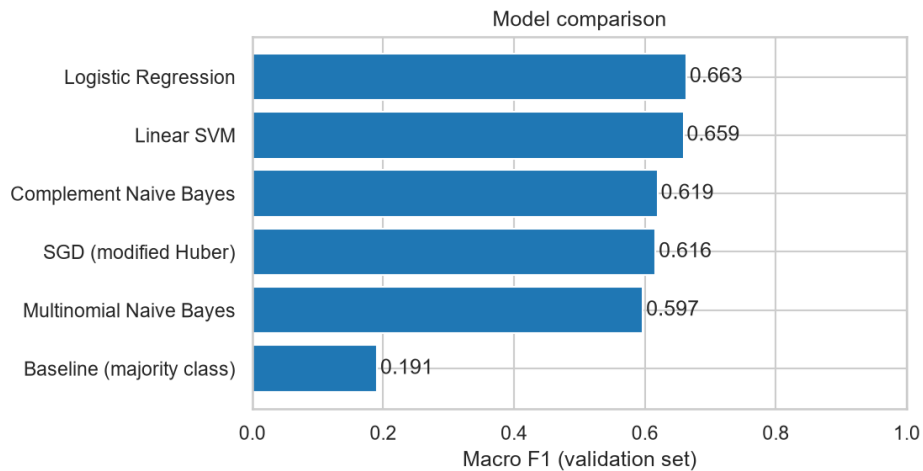


Figure 19: Validation macro-F1 per model

Logistic Regression and the linear SVM lead, followed by Complement Naive Bayes and SGD; every real model beats the baseline by a wide margin. Naive Bayes trains in under a second but gives up some accuracy. The two best models are within a fraction of a point, so Logistic Regression was selected because it also produces calibrated probabilities and directly interpretable coefficients.

Table 20: Tuning the regularisation strength C (validation set)

Model	C	Accuracy	Macro-F1	Weighted-F1
Logistic Regression	0.5	70.1%	0.6666	0.7104
Logistic Regression	1	70.3%	0.6660	0.7112
Logistic Regression	2	70.3%	0.6633	0.7097
Logistic Regression	5	69.8%	0.6542	0.7028
Linear SVM	0.03	71.4%	0.6502	0.7044
Linear SVM	0.1	72.0%	0.6588	0.7121
Linear SVM	0.3	71.5%	0.6555	0.7090
Linear SVM	1	69.8%	0.6412	0.6946

The cross-validated grid search (3 folds, 40,000-review subsample) selected unigrams + bigrams, $\text{min_df} = 5$ and $C = 0.5$ with a best macro-F1 of 0.652. Fine-tuning on the full training set confirmed $C = 0.5$ as best for Logistic Regression: larger values over-fit, which shows as lower validation scores.

8.4 Ablation Study: Do the Design Choices Matter?

Table 21: Ablation on a 60,000-review subsample (validation set)

Setting	Features	Accuracy	Macro-F1
unigrams only	12,971	66.3%	0.6287
unigrams + bigrams (chosen)	58,420	69.0%	0.6494
uni+bi+trigrams	68,327	69.2%	0.6512
bigrams, NO negation kept	53,147	66.3%	0.6231

Setting	Features	Accuracy	Macro-F1
bigrams, raw tf (no sublinear)	58,420	68.9%	0.6491

Adding bigrams improves macro-F1 by about 2 points over unigrams. Removing the negation words (using the standard stop-word list) costs about 2.6 points, which confirms the cleaning decision of Section 5.2. Trigrams add a further 0.2 points at the price of 17 % more features, and sub-linear term frequency matters little here. The chosen configuration is therefore close to the best value for its cost.

8.5 Final Test-set Results

After refitting on train + validation, the model was scored once on the 39,886 held-out test reviews.

Table 22: Test-set performance of the final model

Metric	Value
Accuracy	71.18% (majority baseline: 40.0%)
Macro-F1	0.6757
Weighted-F1	0.7202
ROC-AUC (macro, one-vs-rest)	0.8618

Table 23: Per-class results on the test set

Class	Precision	Recall	F1-score	Support
Negative	0.783	0.738	0.760	15,964
Neutral	0.407	0.509	0.453	7,982
Positive	0.845	0.787	0.815	15,940

Table 24: Confusion matrix (counts)

Actual \ Predicted	Negative	Neutral	Positive	Recall
Negative	11,778	3,414	772	73.8%
Neutral	2,383	4,066	1,533	50.9%
Positive	888	2,506	12,546	78.7%

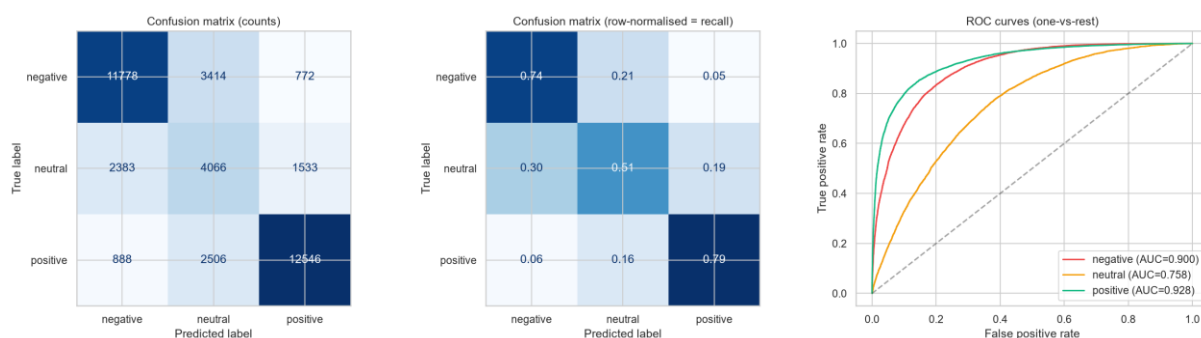


Figure 20: Confusion matrices (counts and row-normalised) and ROC curves on the test set

Negative and positive reviews are recognised with high recall (74% and 79%), and the ROC curves show strong ranking quality (AUC 0.90 for negative, 0.93 for positive). The neutral class is clearly harder: AUC 0.76, F1 0.45. Precision for neutral is low (0.41) because many positive and negative reviews are drawn into the middle class.

8.6 Error Analysis

Of 11,496 test errors, **85.6% involve the neutral class** (either as the true or the predicted label). The most common confusions are: negative → neutral (3,414), positive → neutral (2,506), neutral → negative (2,383), neutral → positive (1,533).

Table 25: Accuracy by review length

Review length (words)	Test reviews	Accuracy
<=10	7,784	75.3%
11-25	13,242	71.7%
26-50	10,994	70.7%
51-100	6,087	67.4%
>100	1,779	64.6%

Very short reviews are the hardest, because a few words give little evidence; accuracy improves with length.

Confident mistakes. The reviews the model got wrong with the highest confidence are instructive:

Table 26: Highest-confidence errors

Review (truncated)	Actual	Predicted	Confidence
All in all, I'm very happy with these. Excellent item, All and all amazing deal for the price! Prompt delivery of a good quality product which is also...	neutral	positive	1.00
Horrible received with scratches,I always support SPIGEN COVERS I WILL NEVER AGAIN.very disappointed 😞	neutral	negative	1.00
I wouldn't recommend.	neutral	negative	0.99
This is great little tool. It can almost any dust or dirt from the car. It is small so my kid can handle it very well and to be good helper. Great buy...	neutral	positive	0.99

Many of these are label noise (a glowing text with a low star rating, usually a mis-clicked rating) or sarcasm. They show a ceiling imposed by the labels rather than a weakness of the algorithm.

8.7 What Did the Model Learn?

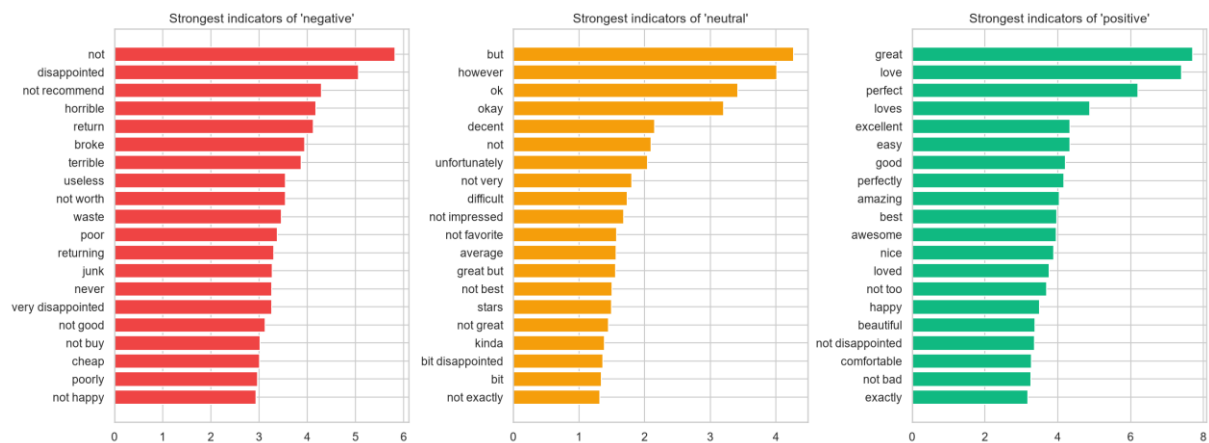


Figure 21: Strongest indicator words per class (largest Logistic Regression coefficients)

- **Negative:** not, disappointed, not recommend, horrible, return, broke, terrible, useless
- **Neutral:** but, however, ok, okay, decent, not, unfortunately, not very
- **Positive:** great, love, perfect, loves, excellent, easy, good, perfectly

The learned words are intuitive, which supports the view that the model has learned genuine sentiment signal rather than artefacts of the dataset.

8.8 Business Insight: Complaint Themes

Reviews were tagged with eight themes by keyword matching, and the share of negative reviews among those mentioning each theme was computed. The overall negative rate is 40 %.

Table 27: Share of negative and positive reviews by theme

Theme	Reviews mentioning	% negative	% positive
Customer service	18,029	69.7%	14.1%
Product quality	30,755	51.5%	28.6%
Shipping / delivery	19,856	51.1%	30.7%
Durability	16,621	49.3%	30.2%
Battery / power	8,841	47.9%	34.9%
Price / value	22,844	40.9%	42.6%
Size / fit	38,676	34.5%	41.6%
Ease of use	19,181	26.7%	55.4%

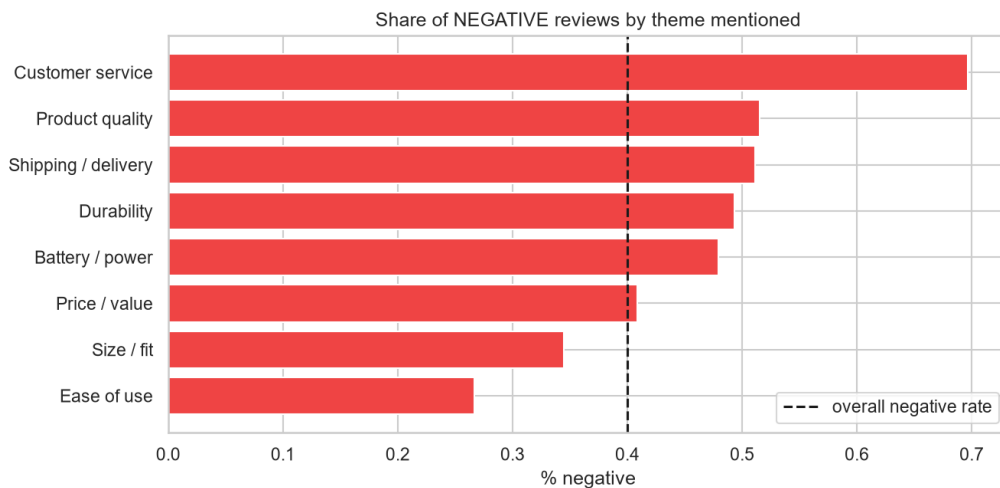


Figure 22: Share of negative reviews by theme (dashed line = overall rate)

Customer service / refunds is by far the strongest predictor of an unhappy customer (70% negative), followed by product quality, shipping and durability. *Ease of use* is the happiest theme (55% positive). The business recommendation is to prioritise support responsiveness and refund handling, and to monitor quality and delivery.

8.9 Performance

Table 28: Measured performance on a laptop (CPU only)

Measure	Value
Single-review prediction (median / mean / 95th percentile)	4.4 ms / 4.5 ms / 5.1 ms
Batch prediction throughput	7,323 reviews per second (39,886 reviews in 5.4 s)
Full training run (python -m src.train)	165 s including six models, tuning and figures
Model file size	5.3 MB

8.10 Threats to Validity and Limitations

- **Noisy labels.** Star ratings are a proxy for sentiment. Mis-clicked ratings and sarcasm cap the achievable accuracy.
- **Neutral ambiguity.** A 3-star review may be mildly positive or mildly negative; even human annotators would disagree.
- **Bag-of-words model.** TF-IDF ignores word order beyond bigrams and cannot resolve sarcasm or long-range negation.
- **Single domain and language.** The data are English Amazon reviews; performance on other domains or languages will differ.
- **Keyword themes.** The theme analysis uses regular expressions, which can over- or under-count topics.
- **Single machine.** The scalability and streaming experiments ran on one laptop; behaviour on a real cluster was not tested.
- **Subsample tuning.** The cross-validated grid search used a 40,000-review subsample for speed.

Chapter 9: Social Impact, Ethics and Sustainability

9.1 Expected Impact

- **Faster feedback loops:** product teams learn about problems within minutes of reviews being posted instead of after manual reading.
- **Better products and less waste:** surfacing durability and quality complaints early reduces returns and discarded goods (SDG 12).
- **Better decisions for small sellers:** the whole pipeline runs on a laptop with open-source tools, lowering the barrier to data-driven management (SDG 9).
- **Transparency:** word-level explanations let a user see why a review was classified as it was.

9.2 Ethical Considerations

- **Privacy.** The dataset has no personal identifiers. Uploaded files are processed in memory only. Any deployment on real customer data must follow data-protection law.
- **Fairness and bias.** The model learns from the language of reviewers who chose to write; it may under-represent groups who write differently. Dialects, slang and non-native English may be misclassified.
- **Human oversight.** Predictions are advisory. Low-confidence results should be reviewed by a person before any action affecting a customer or seller.
- **Misuse.** Sentiment scores must not be used to automatically penalise individual reviewers or to filter out legitimate criticism.
- **Honesty about accuracy.** Reported results are on a held-out set and the limits (especially the neutral class) are stated openly.

9.3 Sustainability

The chosen model trains in seconds on a CPU and the saved file is 5.3 MB, so energy use is a tiny fraction of that of a transformer fine-tuning run. Choosing the simplest model that performs well, and proving it with an ablation, is itself a sustainability decision.

Chapter 10: Mapping to Programme Outcomes and Skills

10.1 Big-data Concepts Demonstrated

Table 29: Big-data concepts demonstrated

Concept	Where it appears in the project
Data collection and profiling	Loading 200k reviews, de-duplication, class balance, length statistics (Sections 5.1, 8.2).
Data cleaning and pre-processing	Custom NLP cleaning routine with negation handling (Section 5.2).
Feature engineering	TF-IDF with n-grams, ablation study (Sections 5.3, 8.4).
Sparse data structures	CSR matrices, memory comparison (Section 6.1).
Scalable machine learning	Linear models, learning curve, timing study (Section 6.2).

Concept	Where it appears in the project
Out-of-core / streaming processing	Chunked reading, hashing trick, incremental SGD (Section 6.3).
Distributed-computing mapping	Spark MLlib equivalents (Section 6.4).
Model evaluation	Validation protocol, macro-F1, ROC-AUC, error analysis (Sections 5.6, 8.5, 8.6).
Data visualisation and storytelling	Notebook figures and interactive Plotly dashboard (Chapter 7).
Business analytics	Complaint-theme analysis and recommendations (Section 8.8).

10.2 Programme Outcome Mapping

Table 30: Programme outcome mapping

PO	Programme outcome	Evidence in the project
PO1	Engineering knowledge	Applies mathematics, statistics and computing (TF-IDF, regression, evaluation metrics) to a real problem.
PO2	Problem analysis	Defines the problem, identifies class imbalance and noisy labels, and analyses errors (Chapters 3, 8).
PO3	Design / development of solutions	Designs the layered architecture and the end-to-end pipeline (Chapters 4, 5).
PO4	Conduct investigations of complex problems	Ablation study, grid search, scalability and streaming experiments (Chapters 6, 8).
PO5	Modern tool usage	Python, scikit-learn, Pandas, Jupyter, Plotly, Streamlit, Git (Chapter 7).
PO6	The engineer and society	Supports consumer information and responsible production (Chapter 9).
PO7	Environment and sustainability	Energy-efficient CPU-only model; SDG 12 link (Section 9.3).
PO8	Ethics	Privacy, fairness, human oversight and honest reporting (Section 9.2).
PO9	Individual and team work	Complete project planned, built, tested and documented independently.
PO10	Communication	Report, presentation script, README and an explanatory dashboard.
PO11	Project management and finance	Staged delivery, reproducible runs, only free open-source tools.
PO12	Life-long learning	Self-learning of big-data techniques such as feature hashing and out-of-core learning.

Chapter 11: Conclusion and Future Scope

11.1 Conclusion

This project set out to classify product reviews as positive, negative or neutral and to do so in a way that stays practical as data grow. A TF-IDF (unigram + bigram) Logistic Regression model, trained on 159,540 reviews and tested on 39,886 unseen ones, reaches **71.2% accuracy, 0.676 macro-F1 and 0.862 ROC-AUC**, well above the 40.0% baseline. Design choices were validated rather than assumed: keeping negations and adding bigrams measurably help, class weights handle the imbalance, and the test set was used only once.

The scalability experiments show that quality keeps improving with more data while cost grows roughly linearly, and the streaming variant shows that the approach does not depend on holding the dataset in memory. The error analysis explains the remaining mistakes: most involve the inherently ambiguous neutral class, and the most confident errors are often label noise. The theme analysis turns the classifier into a business tool by showing that customer service, quality and shipping drive dissatisfaction.

The work also made its limits clear. Star ratings are imperfect labels, a bag-of-words model cannot understand sarcasm, and the experiments ran on a single machine. These are stated openly and addressed in the future scope.

11.2 Future Scope

- Fine-tune a transformer (DistilBERT or RoBERTa) and compare it with the linear baseline on accuracy, cost and energy.
- Add **aspect-based sentiment** (price, quality, delivery, support) to say *what* each review is positive or negative about.
- Replace star-derived labels with human-annotated neutral examples, or model the five-star scale as an ordinal problem.
- Port the pipeline to Apache Spark MLlib or Dask to process millions of reviews on a cluster.
- Add a live data connector, trend monitoring and alerts for sudden drops in sentiment.
- Extend to multilingual reviews and to domain adaptation (electronics, books, food).
- Add model monitoring (drift detection) and a feedback loop where analysts correct predictions.

References

- [1] B. Pang and L. Lee, "Opinion mining and sentiment analysis," *Foundations and Trends in Information Retrieval*, vol. 2, no. 1-2, pp. 1-135, 2008.
- [2] B. Liu, *Sentiment Analysis and Opinion Mining*. San Rafael, CA: Morgan & Claypool, 2012.
- [3] G. Salton and C. Buckley, "Term-weighting approaches in automatic text retrieval," *Information Processing & Management*, vol. 24, no. 5, pp. 513-523, 1988.
- [4] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825-2830, 2011.
- [5] A. McCallum and K. Nigam, "A comparison of event models for naive Bayes text classification," in *AAAI-98 Workshop on Learning for Text Categorization*, 1998, pp. 41-48.

- [6] T. Joachims, "Text categorization with support vector machines: Learning with many relevant features," in Proc. European Conference on Machine Learning (ECML), 1998, pp. 137-142.
- [7] K. Weinberger, A. Dasgupta, J. Langford, A. Smola and J. Attenberg, "Feature hashing for large scale multitask learning," in Proc. 26th International Conference on Machine Learning (ICML), 2009, pp. 1113-1120.
- [8] L. Bottou, "Large-scale machine learning with stochastic gradient descent," in Proc. COMPSTAT, 2010, pp. 177-186.
- [9] J. Devlin, M.-W. Chang, K. Lee and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in Proc. NAACL-HLT, 2019, pp. 4171-4186.
- [10] V. Sanh, L. Debut, J. Chaumond and T. Wolf, "DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter," arXiv:1910.01108, 2019.
- [11] C. Hutto and E. Gilbert, "VADER: A parsimonious rule-based model for sentiment analysis of social media text," in Proc. International AAAI Conference on Web and Social Media (ICWSM), 2014.
- [12] H. He and E. A. Garcia, "Learning from imbalanced data," IEEE Transactions on Knowledge and Data Engineering, vol. 21, no. 9, pp. 1263-1284, 2009.
- [13] M. Zaharia et al., "Apache Spark: A unified engine for big data processing," Communications of the ACM, vol. 59, no. 11, pp. 56-65, 2016.
- [14] T. Fawcett, "An introduction to ROC analysis," Pattern Recognition Letters, vol. 27, no. 8, pp. 861-874, 2006.
- [15] United Nations, "Transforming our world: the 2030 Agenda for Sustainable Development," General Assembly resolution A/RES/70/1, 2015.
- [16] P. Keung, Y. Lu, G. Szarvas and N. A. Smith, "The Multilingual Amazon Reviews Corpus," in Proc. Conference on Empirical Methods in Natural Language Processing (EMNLP), 2020, pp. 4563-4568.
- [17] W. McKinney, "Data structures for statistical computing in Python," in Proc. 9th Python in Science Conference, 2010, pp. 56-61.
- [18] C. R. Harris et al., "Array programming with NumPy," Nature, vol. 585, pp. 357-362, 2020.
- [19] P. Virtanen et al., "SciPy 1.0: fundamental algorithms for scientific computing in Python," Nature Methods, vol. 17, pp. 261-272, 2020.
- [20] D. Jurafsky and J. H. Martin, Speech and Language Processing, 3rd ed. (draft), Ch. 5: Logistic Regression, 2024.

Appendix A: How to Run the Project

Table 31: Commands

Step	Command
Install dependencies	<code>pip install -r requirements.txt</code>
Download the dataset (47 MB)	<code>python -m src.download_data</code>
Train, tune, evaluate and save the model (about 3 minutes)	<code>python -m src.train</code>
Measure latency, memory and the error breakdown	<code>python -m src.facts</code>
Run all automated tests	<code>python -m pytest tests -q</code>
Launch the dashboard	<code>streamlit run app.py</code>
Open the notebook	<code>jupyter lab notebooks/Customer_Review_Sentiment_Analysis.ipynb</code>

Appendix B: Key Source Code

Text cleaning (src/preprocess.py)

```
KEEP_WORDS = {
    "no", "not", "nor", "never", "none", "nothing", "nobody", "neither", "cannot", "without",
    "very", "too", "more", "most", "less", "least", "much", "enough", "but", "however", "although",
}
STOP_WORDS = sorted(ENGLISH_STOP_WORDS - KEEP_WORDS)

_URL = re.compile(r"https?://\S+|www\.\S+")
_TAG = re.compile(r"<[>]+>")
_NON_ALPHA = re.compile(r"[^a-z\s']")
_SPACES = re.compile(r"\s+")
_CONTRACTIONS = [
    (re.compile(r"\bwon't\b"), "will not"),
    (re.compile(r"\bcan't\b"), "can not"),
    (re.compile(r"\bn't\b"), "not"),
    (re.compile(r"'(re|ve|ll|d|m|s)\b"), ""),
]

def clean_text(text: str) -> str:
    """Lower-case, strip HTML/URLs/punctuation/digits, expand negative contractions (don't -> do
    not)."""
    text = html.unescape(str(text)).lower()
    text = _URL.sub(" ", text)
    text = _TAG.sub(" ", text)
    for pattern, repl in _CONTRACTIONS:
        text = pattern.sub(repl, text)
    text = _NON_ALPHA.sub(" ", text).replace("'", "")
    return _SPACES.sub(" ", text).strip()
```

Feature extraction (src/modeling.py)

```
def make_vectorizer(**overrides):
    """TF-IDF over cleaned unigrams+bigrams (sublinear tf, rare/very common terms pruned)."""
    params = dict(preprocessor=clean_text, stop_words=STOP_WORDS, ngram_range=(1, 2), min_df=3,
                  max_df=0.9, max_features=300_000, sublinear_tf=True, dtype=np.float32)
    params.update(overrides)
    return TfidfVectorizer(**params)
```

Model comparison loop (src/train.py)

```
candidates = {
    "Baseline (majority class)": DummyClassifier(strategy="most_frequent"),
    "Multinomial Naive Bayes": MultinomialNB(alpha=0.3),
    "Complement Naive Bayes": ComplementNB(alpha=0.3),
    "SGD (modified Huber)": SGDClassifier(loss="modified_huber", alpha=2e-6,
class_weight="balanced",
                                     max_iter=30, random_state=SEED),
    "Linear SVM": LinearSVC(C=0.1, class_weight="balanced", random_state=SEED),
    "Logistic Regression": LogisticRegression(C=2, class_weight="balanced", max_iter=300),
}
comparison = []
for name, model in candidates.items():
    s = time.time()
    model.fit(Xtr, y_train)
    fit_s = time.time() - s
    comparison.append({"model": name, **scores(y_val, model.predict(Xva)), "train_seconds":
fit_s})
print(f" {name:28s} acc={comparison[-1]['accuracy']:.4f} macroF1={comparison[-
1]['macro_f1']:.4f} ({fit_s:.1f}s)")
```